

Transfer Recipe Selection

This note explains why the transfer stage carries forward two recipes rather than only the single highest mean from the factorial holdout study. The decision is based mainly on `aggregate_report.md` and cross-checked against `novelty_review.md`.

The six factorial candidates were:

`fixed_predator_holdout_endpoint`: trains against one fixed opponent only.

`rotating_opponents_holdout_endpoint`: trains against a rotating pool of opponent types, with no extra archive or novelty filters.

`rotating_plus_nemesis_archive`: trains against rotating opponents and periodically replays opponents that beat the learner earlier.

`rotating_plus_novelty_gate`: trains against rotating opponents and favors candidates that also increase behavioral diversity.

`rotating_plus_replay_aware_selection`: trains against rotating opponents and accepts a new learner only if it survives replay-style checks rather than beating the current opponent once.

`rotating_plus_nemesis_novelty_replay`: combines rotating opponents, nemesis replay, novelty pressure, and replay-aware acceptance.

In the table below, higher is better in every numeric column.

Condition	Transfer status	Held-out win rate	Held-out margin	Learner execution	Fully clean runs
<code>rotating_plus_nemesis_novelty_replay</code>	selected	0.552 (best)	1.520	0.986	1/5
<code>rotating_opponents_holdout_endpoint</code>	selected	0.528	1.608 (best)	0.996	3/5
<code>rotating_plus_replay_aware_selection</code>	not selected	0.520	1.480	0.998 (best)	4/5 (best)
<code>rotating_plus_novelty_gate</code>	not selected	0.536	1.504	0.992	3/5
<code>fixed_predator_holdout_endpoint</code>	not selected	0.472	1.176	0.992	2/5
<code>rotating_plus_nemesis_archive</code>	not selected	0.424	0.408	0.990	3/5

`Held-out win rate` is the primary endpoint. It is the learner's average fraction of wins against the holdout opponent panel, which is evaluated after training and is kept separate from the training opponent pool.

`Held-out margin` is the secondary endpoint. It is the learner's average score minus the opponent's average score on that same holdout panel. It complements win rate by showing how large the advantage was, not just whether the learner won.

`Learner_execution` is the main reliability signal. It is the fraction of epochs in which the learner's submitted code actually executed, rather than falling back because generation or validation failed. `Fully_clean_runs` is a stricter reliability signal: it counts how many replicate runs had zero learner generation errors and zero learner fallback epochs for that condition.

`Fully_clean_runs` is not a separate performance endpoint. It is a caution flag used only when the top-performing conditions are close. A recipe with a slightly better holdout win rate but many fewer clean runs is harder to defend than one with nearly the same holdout result and much cleaner execution.

The first important point is that the top recipes were close. The highest held-out win rate was 0.552, but the next three were 0.536, 0.528, and 0.520. That is not a clean separation. Because the top cluster was narrow, I did not treat the largest single mean as enough by itself. I used four tie-breakers: held-out score margin, reliability, simplicity of interpretation, and the novelty review.

Because the primary endpoint is held-out win rate, `rotating_plus_nemesis_novelty_replay` is carried forward first. Its lead is not large, but it is still the top condition on the main ranking metric. The main caution is reliability: it also had the highest learner fallback count among the leading recipes, which is why its result should be interpreted together with the cleaner comparators rather than in isolation.

`rotating_opponents_holdout_endpoint` is carried forward as the second transfer recipe because it is the strongest simpler reference condition. It had the best held-out score margin in the leading cluster at 1.608, good learner execution at 0.996, and fewer fallback epochs than `rotating_plus_nemesis_novelty_replay`, while still remaining close on the primary endpoint.

The novelty review also matters because it checks whether the biggest code changes look like real strategic change or mostly churn. Across the ten strongest novelty spikes, the review found 0/10 clear cases of behavioral innovation, 6/10 mixed changes, 3/10 minor adjustments, and 1/10 likely code churn. That means the factorial evidence does not justify saying that the novelty-heavier recipes were clearly inventing better strategies. They changed more, but the review does not show strong proof that those changes were better in the deeper sense that matters for transfer.

`rotating_plus_replay_aware_selection` remains a useful follow-up condition because it was the cleanest strong performer, with learner execution 0.998 and 4/5 fully clean runs. It was not included in the first transfer pair because it trailed `rotating_opponents_holdout_endpoint` on both held-out win rate (0.520 vs 0.528) and held-out score margin (1.480 vs 1.608).

These two selected recipes lead to a clear transfer interpretation. If `rotating_plus_nemesis_novelty_replay` transfers better, then the extra memory and selection machinery is helping beyond plain opponent diversity. If `rotating_opponents_holdout_endpoint` transfers similarly or better, then opponent diversity is probably doing most of the useful work and the added machinery may be more specific to the original environment. If neither transfers well, then the phase-2b gains were still too tied to the original resource-collection game.